

EECE 7205 - In-Class Work

Bliss Machado
NUID: 003915475

Problem 1: Bubble Sort

1. Bubble Sort

1.1 Problem Description

Bubble Sort repeatedly steps through the array and compares adjacent elements. If two adjacent elements are in the wrong order, they are swapped. After each pass, the largest remaining element moves to the end of the unsorted part.

The given array is:

34 7 23 32 5 62 14 19

The array needs to be sorted in increasing order using Bubble Sort.

1.2 Approach

I compare adjacent elements from left to right. If the first element is greater than the next element, I swap the two elements.

After each pass, the largest element in the unsorted part moves to the end. I use a **swapped** flag to check if any swap was made during the pass. If no swap is made, the array is already sorted and the program stops.

The program also counts the total number of comparisons and swaps.

1.3 Pseudocode

```
Algorithm BubbleSort(A, n)

comparisons <- 0

swaps <- 0

for i <- 0 to n - 2 do

    swapped <- false

    for j <- 0 to n - 2 - i do

        comparisons <- comparisons + 1

        if A[j] > A[j + 1] then
```

```
        temp <- A[j]

        A[j] <- A[j + 1]

        A[j + 1] <- temp

        swaps <- swaps + 1

        swapped <- true

    end if

end for

printArray(A, n)

if swapped = false then

    break

end if

end for

End Algorithm
```

1.4 Hand Trace

The original array is:

34 7 23 32 5 62 14 19

After the first pass:

7 23 32 5 34 14 19 62

After the second pass:

7 23 5 32 14 19 34 62

After the third pass:

7 5 23 14 19 32 34 62

After the fourth pass:

5 7 14 19 23 32 34 62

After the fifth pass, no swaps are made:

5 7 14 19 23 32 34 62

Therefore, the final sorted array is:

5 7 14 19 23 32 34 62

The program makes 25 comparisons and 15 swaps for the original array.

1.5 Correctness

In each pass, adjacent elements are compared and swapped when they are in the wrong order. This moves the largest element in the unsorted part to the end of that part.

The process continues until no swaps are made. When no swap is made during a complete pass, all adjacent elements are already in the correct order. Therefore, the array is sorted in increasing order.

1.6 Cost Model

Let n be the number of elements in the array.

For an already sorted array, the program makes one pass and $n - 1$ comparisons. Since no swaps are made, the **swapped** flag stops the algorithm early. Therefore, the best case is $O(n)$.

For average and worst case inputs, Bubble Sort takes $O(n^2)$ time.

The algorithm uses a constant amount of extra space, so the extra space is $O(1)$.

Measure	Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Extra Space	$O(1)$

1.7 Already Sorted Copy

The sorted copy of the array is:

5 7 14 19 23 32 34 62

For the already sorted array, the program makes only one pass.

During this pass, each adjacent pair is already in the correct order, so no swaps are made. Therefore, the **swapped** flag remains false and the program stops after the first pass.

The number of comparisons is 7 because there are 8 elements and the first pass compares 7 adjacent pairs.

Measure	Result
Passes	1
Comparisons	7
Swaps	0

1.8 C++ Code

The C++ code is submitted as a separate source file:

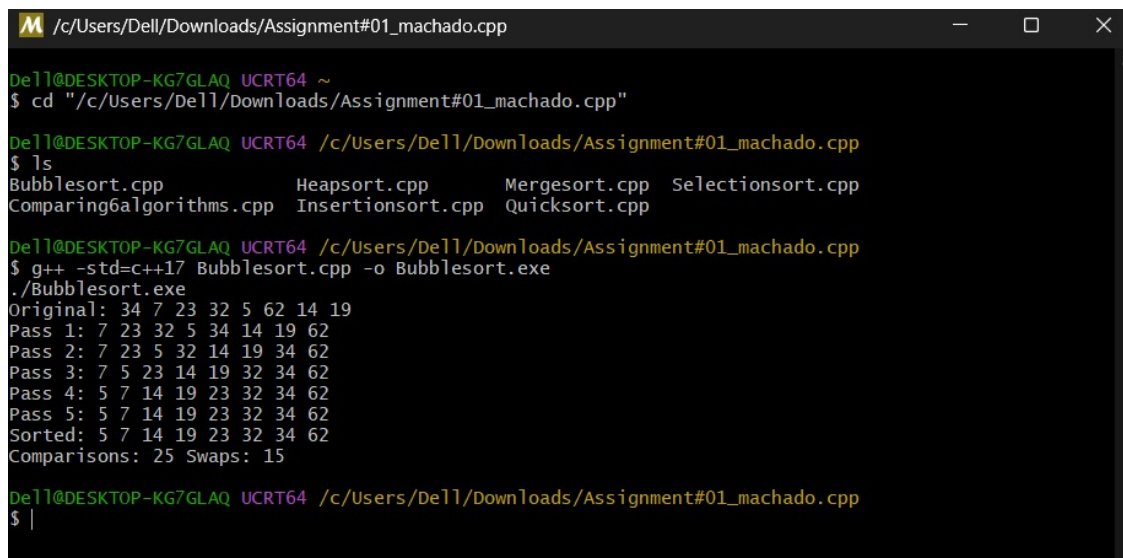
```
Assignment01_Problem01_LastName.cpp
```

The program uses the required `printArray` and `bubbleSort` functions and does not use a built-in sorting function.

1.9 Execution Snapshot

The program was run in VS Code using the given array.

The execution screenshot is shown below.



```
/c/Users/Dell/Downloads/Assignment#01_machado.cpp
Dell@DESKTOP-KG7GLAQ UCRT64 ~
$ cd "/c/Users/Dell/Downloads/Assignment#01_machado.cpp"

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ ls
Bubblesort.cpp          Heapsort.cpp          Mergesort.cpp          Selectionsort.cpp
Comparing6algorithms.cpp Insertionsort.cpp      Quicksort.cpp

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ g++ -std=c++17 Bubblesort.cpp -o Bubblesort.exe
./Bubblesort.exe
Original: 34 7 23 32 5 62 14 19
Pass 1: 7 23 32 5 34 14 19 62
Pass 2: 7 23 5 32 14 19 34 62
Pass 3: 7 5 23 14 19 32 34 62
Pass 4: 5 7 14 19 23 32 34 62
Pass 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 25 Swaps: 15

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ |
```

Figure 1: Output 1

2. Insertion Sort

2.1 Problem Description

Insertion Sort builds the sorted part of the array one element at a time. It takes the next element as the key and shifts every larger element in the sorted part one position to the right until the key can be inserted in its correct position.

The given array is:

34 7 23 32 5 62 14 19

The array needs to be sorted in increasing order using Insertion Sort.

2.2 Approach

I start from the second element because the first element is already considered sorted. For each step, I take the current element as the key and compare it with the elements before it.

If an element is larger than the key, I shift it one position to the right. I continue this until the correct position for the key is found, and then I place the key there.

The program also counts the total number of comparisons and shifts.

2.3 Pseudocode

```
Algorithm InsertionSort(A, n)

comparisons <- 0

shifts <- 0

for i <- 1 to n - 1 do
    key <- A[i]
    j <- i - 1
    while j >= 0 do
        comparisons <- comparisons + 1
        if A[j] > key then
            A[j + 1] <- A[j]
            shifts <- shifts + 1
            j <- j - 1
```

```

        else
            break
        end if
    end while
A[j + 1] <- key
printArray(A, n)
end for
End Algorithm

```

2.4 Hand Trace

The original array is:

34 7 23 32 5 62 14 19

For $i = 1$, the key is 7:

7 34 23 32 5 62 14 19

For $i = 2$, the key is 23:

7 23 34 32 5 62 14 19

For $i = 3$, the key is 32:

7 23 32 34 5 62 14 19

For $i = 4$, the key is 5:

5 7 23 32 34 62 14 19

For $i = 5$, the key is 62:

5 7 23 32 34 62 14 19

For $i = 6$, the key is 14:

5 7 14 23 32 34 62 19

For $i = 7$, the key is 19:

5 7 14 19 23 32 34 62

Therefore, the final sorted array is:

5 7 14 19 23 32 34 62

For the original array, the program makes 20 comparisons and 15 shifts.

2.5 Correctness

At the beginning of each step, the elements before the key are already sorted.

If an element is larger than the key, it is shifted one position to the right. This continues until the key can be placed in its correct position.

After inserting the key, the sorted part becomes one element larger. Therefore, after all elements have been processed, the whole array is sorted in increasing order.

2.6 Cost Model

For Insertion Sort, I count the number of comparisons and shifts made while inserting each key.

Step	Comparisons	Shifts	height
1	1 $i = 2$	2	1
1 $i = 3$	2	1 $i = 4$	2
4	4 $i = 5$	1	3
0 $i = 6$	5	4 $i = 7$	4
5	4 height	Total	20
15 height			

The total number of shifts for the given array is 15.

2.7 Fewest and Most Shifts

The same eight numbers make the fewest shifts when they are already sorted:

5 7 14 19 23 32 34 62

In this case, no element needs to be shifted, so the number of shifts is 0.

The same eight numbers make the most shifts when they are in decreasing order:

62 34 32 23 19 14 7 5

In this ordering, every new key has to be moved to the beginning of the sorted part. For 8 elements, the number of shifts is:

$$7 + 6 + 5 + 4 + 3 + 2 + 1 = 28$$

Therefore, the fewest number of shifts is 0 and the most number of shifts is 28.

2.8 Execution Snapshot

The program was run in VS Code using the given array and the two additional orderings.

The execution screenshots are shown below.

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ g++ -std=c++17 Insertionsort.cpp -o Insertionsort.exe
./Insertionsort.exe
Original: 34 7 23 32 5 62 14 19
i = 1, key = 7: 7 34 23 32 5 62 14 19
i = 2, key = 23: 7 23 34 32 5 62 14 19
i = 3, key = 32: 7 23 32 34 5 62 14 19
i = 4, key = 5: 5 7 23 32 34 62 14 19
i = 5, key = 62: 5 7 23 32 34 62 14 19
i = 6, key = 14: 5 7 14 23 32 34 62 19
i = 7, key = 19: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 20 Shifts: 15

```

Figure 2: Output 1

```

Already sorted: 5 7 14 19 23 32 34 62
i = 1, key = 7: 5 7 14 19 23 32 34 62
i = 2, key = 14: 5 7 14 19 23 32 34 62
i = 3, key = 19: 5 7 14 19 23 32 34 62
i = 4, key = 23: 5 7 14 19 23 32 34 62
i = 5, key = 32: 5 7 14 19 23 32 34 62
i = 6, key = 34: 5 7 14 19 23 32 34 62
i = 7, key = 62: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 7 Shifts: 0

Reversed: 62 34 32 23 19 14 7 5
i = 1, key = 34: 34 62 32 23 19 14 7 5
i = 2, key = 32: 32 34 62 23 19 14 7 5
i = 3, key = 23: 23 32 34 62 19 14 7 5
i = 4, key = 19: 19 23 32 34 62 14 7 5
i = 5, key = 14: 14 19 23 32 34 62 7 5
i = 6, key = 7: 7 14 19 23 32 34 62 5
i = 7, key = 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Shifts: 28

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp

```

Figure 3: Output 2

3. Selection Sort

3.1 Problem Description

Selection Sort divides the array into a sorted part on the left and an unsorted part on the right. In each pass, it finds the smallest element in the unsorted part and swaps it into the first position of the unsorted part.

The given array is:

34 7 23 32 5 62 14 19

The array needs to be sorted in increasing order using Selection Sort.

3.2 Approach

I start from the first position and search the remaining unsorted part of the array for the smallest element.

After finding the minimum element, I swap it with the element at the current position. The sorted part becomes one element larger after each pass.

I also count the number of comparisons and swaps made by the algorithm.

3.3 Pseudocode

```
Algorithm SelectionSort(A, n)

comparisons <- 0

swaps <- 0

for i <- 0 to n - 2 do
    minIndex <- i
    for j <- i + 1 to n - 1 do
        comparisons <- comparisons + 1
        if A[j] < A[minIndex] then
            minIndex <- j
        end if
    end for
end for

if minIndex != i then
```

```

    temp <- A[i]

    A[i] <- A[minIndex]

    A[minIndex] <- temp

    swaps <- swaps + 1

end if

print minIndex and A[i]

printArray(A, n)

end for

End Algorithm

```

3.4 Hand Trace

The original array is:

```
34 7 23 32 5 62 14 19
```

Pass 1: the minimum is 5 at index 4.

```
5 7 23 32 34 62 14 19
```

Pass 2: the minimum is 7 at index 1.

```
5 7 23 32 34 62 14 19
```

Pass 3: the minimum is 14 at index 6.

```
5 7 14 32 34 62 23 19
```

Pass 4: the minimum is 19 at index 7.

```
5 7 14 19 34 62 23 32
```

Pass 5: the minimum is 23 at index 6.

```
5 7 14 19 23 62 34 32
```

Pass 6: the minimum is 32 at index 7.

```
5 7 14 19 23 32 34 62
```

Pass 7: the minimum is 34 at index 6.

```
5 7 14 19 23 32 34 62
```

Therefore, the final sorted array is:

```
5 7 14 19 23 32 34 62
```

3.5 Correctness

In each pass, the algorithm searches the unsorted part and finds its smallest element. That element is placed at the beginning of the unsorted part.

After each pass, one more element is in its correct position. After all the passes are complete, all elements are in increasing order.

3.6 Cost Model

For Selection Sort, I count the comparisons made while searching for the minimum and the swaps made when the minimum is placed in its correct position.

Pass	Minimum	Comparisons	Swaps
5 at index 4	7	1 2	7 at index 1
6	0 3	14 at index 6	5
1 4	19 at index 7	4	1 5
23 at index 6	3	1 6	32 at index 7
2	1 7	34 at index 6	1
0 height		28	5 height
Total			

There are 7 comparisons in the first pass, then 6, then 5, and so on.

Therefore, the total number of comparisons is:

$$7 + 6 + 5 + 4 + 3 + 2 + 1 = 28$$

There are 5 actual swaps because on Pass 2 and Pass 7 the minimum element is already in the correct position.

3.7 Comparison with Bubble Sort

For the given array, Bubble Sort made 15 swaps.

Selection Sort makes 5 swaps on the same array.

Therefore, Selection Sort makes fewer swaps for this input. Selection Sort searches for the minimum first and then moves it into position with one swap, instead of repeatedly swapping adjacent elements as Bubble Sort does.

3.8 Execution Snapshot

The program was run in VS Code using the given array.

The execution screenshots are shown below.

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ g++ -std=c++17 Selectionsort.cpp -o Selectionsort.exe
./Selectionsort.exe
Original: 34 7 23 32 5 62 14 19
Pass 1: min = 5 at index 4 -> 5 7 23 32 34 62 14 19
Pass 2: min = 7 at index 1 -> 5 7 23 32 34 62 14 19
Pass 3: min = 14 at index 6 -> 5 7 14 32 34 62 23 19
Pass 4: min = 19 at index 7 -> 5 7 14 19 34 62 23 32
Pass 5: min = 23 at index 6 -> 5 7 14 19 23 62 34 32
Pass 6: min = 32 at index 7 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Swaps: 5

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$

```

Figure 4: Output 1

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ g++ -std=c++17 Mergesort.cpp -o Mergesort.exe
./Mergesort.exe
Original: 34 7 23 32 5 62 14 19
split: 34 7 23 32 5 62 14 19
split: 34 7 23 32
split: 34 7
merged: 7 34
split: 23 32
merged: 23 32
merged: 7 23 32 34
split: 5 62 14 19
split: 5 62
merged: 5 62
split: 14 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 17 (final merge: 7)

```

Figure 5: Out 2

```
Already sorted: 5 7 14 19 23 32 34 62
split: 5 7 14 19 23 32 34 62
split: 5 7 14 19
split: 5 7
merged: 5 7
split: 14 19
merged: 14 19
merged: 5 7 14 19
split: 23 32 34 62
split: 23 32
merged: 23 32
split: 34 62
merged: 34 62
merged: 23 32 34 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)

Reversed: 62 34 32 23 19 14 7 5
split: 62 34 32 23 19 14 7 5
split: 62 34 32 23
split: 62 34
merged: 34 62
split: 32 23
merged: 23 32
merged: 23 32 34 62
split: 19 14 7 5
split: 19 14
merged: 14 19
split: 7 5
merged: 5 7
merged: 5 7 14 19
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
```

Figure 6: Output 2

4. Quick Sort

4.1 Problem Description

Quick Sort is a divide-and-conquer sorting algorithm. It chooses a pivot and partitions the array so that smaller elements are placed on the left side of the pivot and larger elements are placed on the right side.

For this problem, the last element of the array is used as the pivot using the Lomuto partition method.

The given array is:

34 7 23 32 5 62 14 19

The array needs to be sorted in increasing order using Quick Sort.

4.2 Approach

I use the last element as the pivot. During partitioning, I compare each element with the pivot.

If the element is smaller than or equal to the pivot, I move it to the left side of the pivot.

After all the elements have been checked, I place the pivot in its final position.

Quick Sort is then called recursively on the left and right parts of the array.

I also count the total number of comparisons made during partitioning.

4.3 Pseudocode

```
Algorithm QuickSort(A, low, high)

if low < high then
    '''
    pivotIndex <- Partition(A, low, high)

    QuickSort(A, low, pivotIndex - 1)

    QuickSort(A, pivotIndex + 1, high)
    '''

end if

End Algorithm
```

Partition

Algorithm Partition(A, low, high)

```

pivot <- A[high]

i <- low - 1

for j <- low to high - 1 do
  ''
  comparisons <- comparisons + 1
  if A[j] <= pivot then
    i <- i + 1
    temp <- A[i]
    A[i] <- A[j]
    A[j] <- temp
  end if
end for

temp <- A[i + 1]
A[i + 1] <- A[high]
A[high] <- temp

return i + 1

End Algorithm

```

4.4 Hand Trace

The original array is:

34 7 23 32 5 62 14 19

The first pivot is the last element:

Pivot = 19

The elements smaller than or equal to 19 are moved to the left side.

After the first partition:

7 5 14 19 34 62 23 32

The pivot 19 is now at index 3.

The left part is:

7 5 14

The right part is:

34 62 23 32

For the left part, the pivot is 14:

7 5 14

After partitioning:

7 5 14

The pivot 14 is at its final position.

The remaining left part is:

7 5

Using 5 as the pivot gives:

5 7

For the right part of the original array, the pivot is 32:

34 62 23 32

After partitioning:

23 32 34 62

The pivot 32 is now in its final position.

The remaining elements are already in the correct order:

34 62

Therefore, the final sorted array is:

5 7 14 19 23 32 34 62

4.5 Correctness

In each partition, elements smaller than or equal to the pivot are placed on its left side. The larger elements are placed on its right side.

After the pivot is placed in its final position, it does not need to be moved again.

Quick Sort then applies the same process to the left and right subarrays.

When the subarray contains zero or one element, it is already sorted.

Therefore, after all recursive calls are completed, the complete array is sorted in increasing order.

4.6 Cost Model

For the original array, the comparisons made during each partition are:

Pivot	Subarray Size	Comparisons	height
8	7	3	19
2	5	2	32
4	3	1	32
13	height	Total	

The total number of comparisons is therefore:

$$7 + 2 + 1 + 3 = 13$$

The final sorted array is:

5	7	14	19	23	32	34	62
---	---	----	----	----	----	----	----

4.7 Already Sorted Copy

The already sorted copy of the array is:

5	7	14	19	23	32	34	62
---	---	----	----	----	----	----	----

For this input, the last element is always the largest element in the current subarray.

Therefore, every partition places the pivot at the end of the current subarray. The problem size is reduced by only one element each time.

The number of comparisons is:

$$7 + 6 + 5 + 4 + 3 + 2 + 1 = 28$$

Therefore, the sorted copy makes 28 comparisons.

This is the worst case for Quick Sort when the last element is always chosen as the pivot because the partitions are very unbalanced.

One way to avoid this is to choose a different pivot, such as the middle element or a randomly selected element.

Input	Comparisons	height
13	28	height
Already sorted array		

4.8 Execution Snapshot

The program was run in VS Code using the given array.

The execution screenshots are shown below.

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ g++ -std=c++17 Quicksort.cpp -o Quicksort.exe
./Quicksort.exe
Original: 34 7 23 32 5 62 14 19
pivot = 19: 34 7 23 32 5 62 14 19
i = -1, j = 0
i = -1, j = 1
After swap: 7 34 23 32 5 62 14 19
i = 0, j = 2
i = 0, j = 3
i = 0, j = 4
After swap: 7 5 23 32 34 62 14 19
i = 1, j = 5
i = 1, j = 6
After swap: 7 5 14 32 34 62 23 19
Pivot final position: 3
7 5 14 19 34 62 23 32
pivot = 19: 7 5 14 19 34 62 23 32
pivot = 14: 7 5 14
  pivot = 14: 7 5 14
pivot = 5: 7 5
  pivot = 5: 5 7
pivot = 32: 34 62 23 32
  pivot = 32: 23 32 34 62
pivot = 62: 34 62
  pivot = 62: 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 14

```

Figure 7: Output 1

```

Already sorted: 5 7 14 19 23 32 34 62
pivot = 62: 5 7 14 19 23 32 34 62
i = -1, j = 0
After swap: 5 7 14 19 23 32 34 62
i = 0, j = 1
After swap: 5 7 14 19 23 32 34 62
i = 1, j = 2
After swap: 5 7 14 19 23 32 34 62
i = 2, j = 3
After swap: 5 7 14 19 23 32 34 62
i = 3, j = 4
After swap: 5 7 14 19 23 32 34 62
i = 4, j = 5
After swap: 5 7 14 19 23 32 34 62
i = 5, j = 6
After swap: 5 7 14 19 23 32 34 62
Pivot final position: 7
5 7 14 19 23 32 34 62
pivot = 62: 5 7 14 19 23 32 34 62
pivot = 34: 5 7 14 19 23 32 34
  pivot = 34: 5 7 14 19 23 32 34
pivot = 32: 5 7 14 19 23 32
  pivot = 32: 5 7 14 19 23 32
pivot = 23: 5 7 14 19 23
  pivot = 23: 5 7 14 19 23
pivot = 19: 5 7 14 19
  pivot = 19: 5 7 14 19
pivot = 14: 5 7 14
  pivot = 14: 5 7 14
pivot = 7: 5 7
  pivot = 7: 5 7
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp

```

Figure 8: Output 2

5. Merge Sort

5.1 Problem Description

Merge Sort is a divide-and-conquer sorting algorithm. It splits the array into two halves, sorts each half recursively, and then merges the two sorted halves into one sorted array.

For this problem, the given array is:

34 7 23 32 5 62 14 19

The array is sorted in increasing order using Merge Sort.

The midpoint is calculated using:

$$mid = \frac{low + high}{2}$$

5.2 Approach

I first divide the array into two halves.

I continue dividing each half until each subarray contains one element.

Then I merge the smaller sorted subarrays together.

During each merge, I compare the elements from the two sorted halves and place the smaller element into the temporary array.

After the merge is complete, the values are copied back into the original array.

I also count the comparisons made while merging.

5.3 Pseudocode

```
Algorithm MergeSort(A, low, high)

if low < high then

mid <- (low + high) / 2

MergeSort(A, low, mid)

MergeSort(A, mid + 1, high)

Merge(A, low, mid, high)
end if

End Algorithm
```

Merge

```

Algorithm Merge(A, low, mid, high)

create temporary array

i <- low

j <- mid + 1

k <- 0

while i <= mid and j <= high do
  comparisons <- comparisons + 1

  if A[i] <= A[j] then
    temp[k] <- A[i]
    i <- i + 1
  else
    temp[k] <- A[j]
    j <- j + 1
  end if

  k <- k + 1
end while

while i <= mid do
  temp[k] <- A[i]
  i <- i + 1
  k <- k + 1
end while

while j <= high do
  temp[k] <- A[j]
  j <- j + 1

```

```

k <- k + 1

end while

copy temp back into A

End Algorithm

```

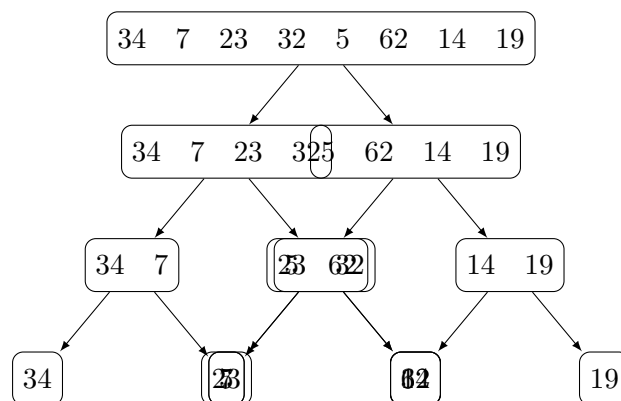
5.4 Hand Trace

The original array is:

```
34 7 23 32 5 62 14 19
```

Splitting the Array

The complete splitting process is shown below.



The array has now been divided into single-element subarrays.

Merging the Left Half

First, the pairs are merged:

$$\{34\}, \{7\} \longrightarrow \{7, 34\}$$

$$\{23\}, \{32\} \longrightarrow \{23, 32\}$$

The two sorted subarrays are then merged:

$$\{7, 34\}, \{23, 32\} \longrightarrow \{7, 23, 32, 34\}$$

The left half is now sorted.

Merging the Right Half

The pairs are merged first:

$$\{5\}, \{62\} \longrightarrow \{5, 62\}$$

$$\{14\}, \{19\} \longrightarrow \{14, 19\}$$

The two sorted subarrays are then merged:

$$\{5, 62\}, \{14, 19\} \longrightarrow \{5, 14, 19, 62\}$$

The right half is now sorted.

Final Merge

The two sorted halves are:

$$\{7, 23, 32, 34\} \quad \{5, 14, 19, 62\}$$

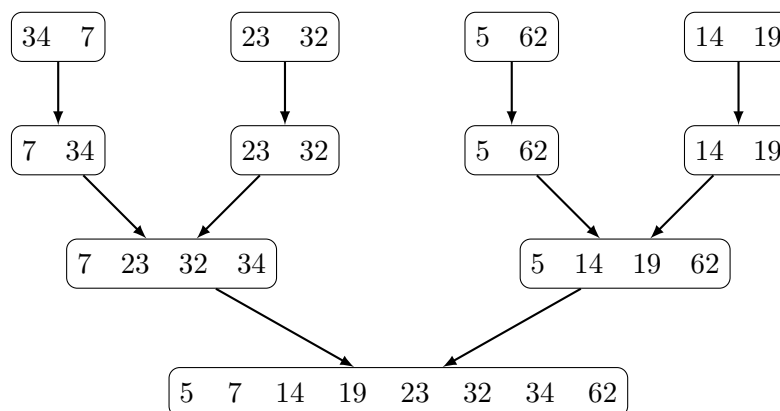
They are merged as follows:

Left	Right	Selected
7	5	5
7	14	7
23	14	14
23	19	19
23	62	23
32	62	32
34	62	34

After the final merge:

5	7	14	19	23	32	34	62
---	---	----	----	----	----	----	----

The complete merge process can be represented as:



5.5 Correctness

Merge Sort divides the array into smaller subarrays until each subarray contains one element.

A single-element subarray is already sorted.

During merging, the smaller element from the two sorted subarrays is selected first. Therefore, each merged subarray is sorted.

The same process is repeated until the two sorted halves are merged into the complete array.

Therefore, the final array is sorted in increasing order.

5.6 Cost Model

For the original array, the comparisons during each merge are:

Merge	Comparisons
34 and 7	1
23 and 32	1
7, 34 and 23, 32	3
5 and 62	1
14 and 19	1
5, 62 and 14, 19	3
7, 23, 32, 34 and 5, 14, 19, 62	7
Total	17

The final merge makes 7 comparisons.

Therefore:

$$\text{Total comparisons} = 17$$

$$\text{Final merge comparisons} = 7$$

Available next action: :contentReference[oaicite:0]index=0

5.7 Sorted and Reversed Copies

The already sorted copy is:

5 7 14 19 23 32 34 62

The reversed copy is:

62 34 32 23 19 14 7 5

For the sorted copy, the number of comparisons is 12.

For the reversed copy, the number of comparisons is also 12.

The number of comparisons changes much less than with Bubble Sort or Quick Sort because Merge Sort always divides the array into halves in the same way. The merging

process still has to combine the sorted subarrays even when the input is already sorted or reversed.

Merge Sort also requires extra memory for the temporary arrays used during merging. For this implementation, the temporary array is used during each merge.

Input	Comparisons
17 heightAlready sorted array	heightOriginal array
12 height	heightReversed array

5.8 Execution Snapshot

The program was run in VS Code using the given array.

The execution screenshots are shown below.

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ g++ -std=c++17 Mergesort.cpp -o Mergesort.exe && ./Mergesort.exe
Original: 34 7 23 32 5 62 14 19
split: 34 7 23 32 5 62 14 19
split: 34 7 23 32
split: 34 7
split: 34
merged: 7 34
split: 23 32
merged: 23 32
merged: 7 23 32 34
split: 5 62 14 19
split: 5 62
merged: 5 62
split: 14 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 17 (final merge: 7)

```

Figure 9: Output 1

```

Already sorted: 5 7 14 19 23 32 34 62
split: 5 7 14 19 23 32 34 62
split: 5 7 14 19
split: 5 7
merged: 5 7
split: 14 19
merged: 14 19
merged: 5 7 14 19
split: 23 32 34 62
split: 23 32
merged: 23 32
split: 34 62
merged: 34 62
merged: 23 32 34 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)

Reversed: 62 34 32 23 19 14 7 5
split: 62 34 32 23 19 14 7 5
split: 62 34 32 23
split: 62 34
merged: 34 62
split: 32 23
merged: 23 32
merged: 23 32 34 62
split: 19 14 7 5
split: 19 14
merged: 14 19
split: 7 5
merged: 5 7
merged: 5 7 14 19
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ |

```

Figure 10: Output 2

6. Heap Sort

6.1 Description

Heap Sort uses a binary max-heap to sort the array in increasing order. In a max-heap, every parent is greater than or equal to its children.

For an element at index k , the left child is at index $2k + 1$ and the right child is at index $2k + 2$.

First, the array is changed into a max-heap. The largest value is then at the root of the heap. The root is swapped with the last element of the unsorted part, and the remaining heap is restored using the sift-down operation.

This process continues until the complete array is sorted in increasing order.

6.2 Approach

The array is first converted into a max-heap.

Starting from the last parent node, the sift-down operation is applied to each parent until the root is reached. This makes the largest value move to the root.

After the max-heap is built, the root is swapped with the last element of the unsorted part. The heap size is then reduced by one, and sift-down is used again to restore the max-heap property.

The process is repeated until only one element remains in the heap.

The input array is:

$$A = [34, 7, 23, 32, 5, 62, 14, 19]$$

6.3 Pseudocode

```
Algorithm HeapSort(A, n)

swaps <- 0

for root <- n/2 - 1 down to 0 do
    siftDown(A, root, n)
end for

printArray(A, n)

for end <- n - 1 down to 1 do
    temp <- A[0]
    A[0] <- A[end]
```

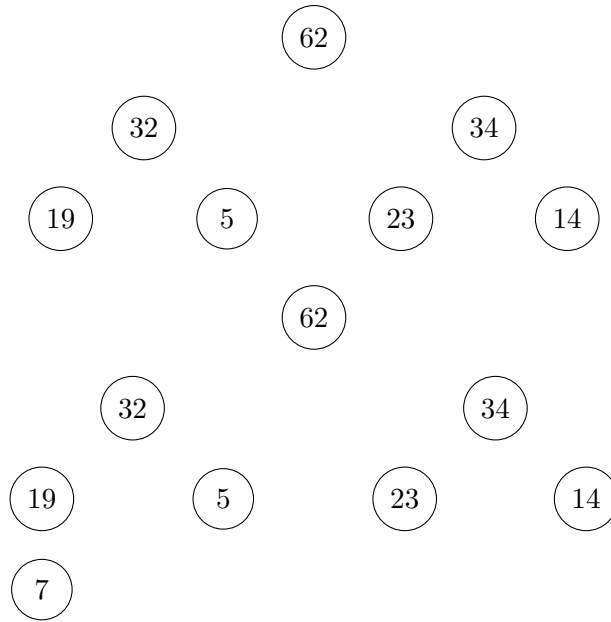
```

    A[end] <- temp
    siftDown(A, 0, end)
    printArray(A, end)
    printArray(A + end, n - end)
end for
End Algorithm

Algorithm siftDown(A, root, size)
while 2 * root + 1 < size do
    child <- 2 * root + 1
    if child + 1 < size and A[child + 1] > A[child] then
        child <- child + 1
    end if
    if A[root] >= A[child] then
        return
    end if
    temp <- A[root]
    A[root] <- A[child]
    A[child] <- temp
    swaps <- swaps + 1
    root <- child
end while
End Algorithm

```

The max-heap after the build-max-heap phase is:



6.6 Cost Model

For the original array, the swaps made inside `siftDown` are:

Phase	Sift-down Swaps
Build max-heap	4
end = 7	2
end = 6	2
end = 5	1
end = 4	2
end = 3	1
end = 2	0
end = 1	0
Total	12

The swap that moves the root to the end of the array is not included because the question asks for the swaps made inside `siftDown`.

Therefore:

$$\boxed{\text{Swaps in siftDown} = 12}$$

6.9 Stability

Heap Sort is not stable.

For example, consider the array:

$$[4_A, 4_B, 3]$$

Here, 4_A and 4_B have the same value, but they represent two different elements.

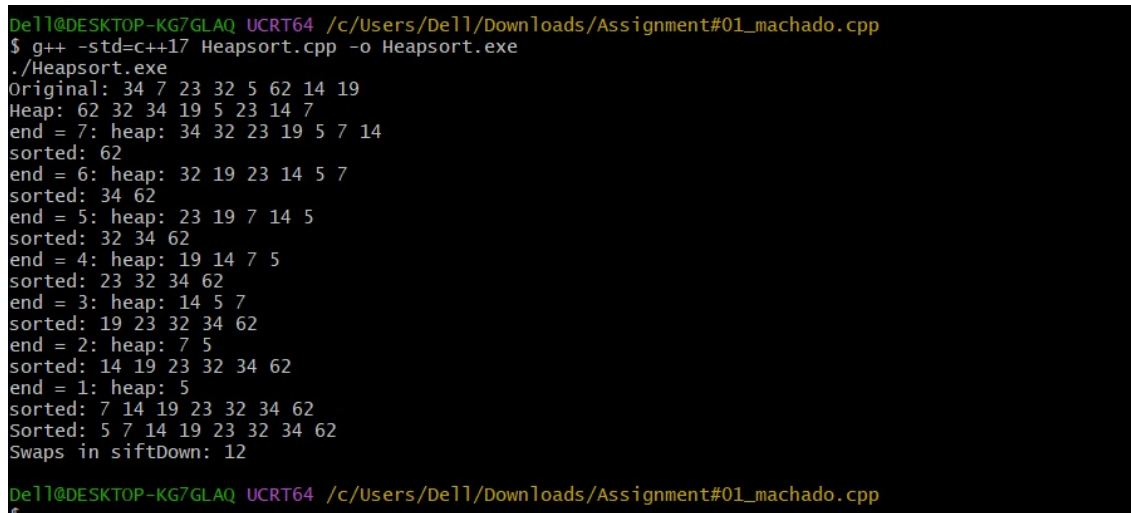
During Heap Sort, elements are moved using swaps based on their positions in the heap. Equal values can change their relative order during these swaps.

Therefore, even though the two values are both 4, the order $4_A, 4_B$ is not guaranteed to remain the same.

Heap Sort is not stable.

6.10 Execution Snapshot

The following screenshot shows the output of my C++ program for Heap Sort.



```
De11@DESKTOP-KG7GLAQ UCR764 /c/Users/De11/Downloads/Assignment#01_machado.cpp
$ g++ -std=c++17 Heapsort.cpp -o Heapsort.exe
./Heapsort.exe
Original: 34 7 23 32 5 62 14 19
Heap: 62 32 34 19 5 23 14 7
end = 7: heap: 34 32 23 19 5 7 14
sorted: 62
end = 6: heap: 32 19 23 14 5 7
sorted: 34 62
end = 5: heap: 23 19 7 14 5
sorted: 32 34 62
end = 4: heap: 19 14 7 5
sorted: 23 32 34 62
end = 3: heap: 14 5 7
sorted: 19 23 32 34 62
end = 2: heap: 7 5
sorted: 14 19 23 32 34 62
end = 1: heap: 5
sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Swaps in siftDown: 12
De11@DESKTOP-KG7GLAQ UCR764 /c/Users/De11/Downloads/Assignment#01_machado.cpp
```

Figure 11: Output1

7. Comparing

1. Problem Description

The purpose of this problem is to compare Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, and Heap Sort.

The program uses the same array from the previous problems:

$$A = \{34, 7, 23, 32, 5, 62, 14, 19\}$$

The program counts comparisons on the original array, a sorted copy, and a reversed copy. It also measures the running time of the six algorithms on random arrays of sizes 1,000, 5,000, and 10,000.

The step-by-step printing used in Problems 1–6 is turned off for this problem. The program still uses the required `printArray` function.

2. Pseudocode

2.1 Main Program

```
Algorithm CompareSixAlgorithms

Create array A

Create a sorted copy of A
Create a reversed copy of A

For each sorting algorithm

    Set comparisons to 0

    Sort A
    Record comparisons

    Set comparisons to 0

    Sort sorted copy
    Record comparisons

    Set comparisons to 0

    Sort reversed copy
    Record comparisons

End for
```

```

Create array sizes 1000, 5000, and 10000

For each array size

    Generate random values between 1 and 100000

    For each sorting algorithm

        Copy the same random array into a new array

        Start the timer

        Sort the new array

        Stop the timer

        Record the running time

        Print the first 10 elements

    End for

End for

End Algorithm

```

2.2 Bubble Sort

```

Algorithm BubbleSort(arr, n)

For i = 0 to n - 2

    Set swapped to false

    For j = 0 to n - 2 - i

        Count one comparison

        If arr[j] > arr[j + 1] then

            Swap arr[j] and arr[j + 1] using a temporary variable

            Set swapped to true

        End if

    End for

End for

```

```
End for

If swapped is false then
    Stop the algorithm
End if

End for

End Algorithm
```

2.3 Insertion Sort

```
Algorithm InsertionSort(arr, n)

For i = 1 to n - 1
    key = arr[i]
    j = i - 1
    While j >= 0
        Count one comparison
        If arr[j] > key then
            Move arr[j] one position to the right
            Decrease j by 1
        Else
            Stop the while loop
        End if
    End while
    Put key in arr[j + 1]
End for

End Algorithm
```

2.4 Selection Sort

```
Algorithm SelectionSort(arr, n)
For i = 0 to n - 2
    Set minIndex = i
    For j = i + 1 to n - 1
        Count one comparison
        If arr[j] < arr[minIndex] then
            Set minIndex = j
        End if
    End for
    Swap arr[i] and arr[minIndex] using a temporary variable
End for
End Algorithm
```

2.5 Quick Sort

```
Algorithm QuickSort(arr, low, high)
If low < high then
    Choose the last element as the pivot
    Partition the array around the pivot
    Recursively sort the left part
    Recursively sort the right part
End if
End Algorithm
```

2.6 Merge Sort

```
Algorithm MergeSort(arr, low, high)
If low < high then
    mid = (low + high) / 2
    Sort the left half
    Sort the right half
    Merge the two sorted halves
End if
End Algorithm
```

2.7 Heap Sort

```
Algorithm HeapSort(arr, n)
Build a max heap
For end = n - 1 down to 1
    Swap the root with arr[end]
    Restore the max heap using siftDown
End for
End Algorithm
```

3. Comparison Counts

The following table shows the number of comparisons made by each algorithm on the original array, the sorted copy, and the reversed copy.

The counts depend on how comparisons are counted in the program. In this program, a comparison counter is increased at the comparison performed by each algorithm.

4. Timing Results

The program generates random integers between 1 and 100,000 for arrays of size 1,000, 5,000, and 10,000.

Algorithm	Original A	Sorted A	Reversed A
Bubble Sort	25	7	28
Insertion Sort	20	7	28
Selection Sort	28	28	28
Quick Sort	14	28	28
Merge Sort	17	12	12
Heap Sort	30	32	28

Table 1: Number of comparisons for the three inputs.

For each array size, the same random data is copied into a new array before each algorithm is run. This makes the timing comparison use the same input data for all six algorithms.

The running time is measured using `<chrono>`.

Algorithm	1,000	5,000	10,000
Bubble Sort	—	—	—
Insertion Sort	—	—	—
Selection Sort	—	—	—
Quick Sort	—	—	—
Merge Sort	—	—	—
Heap Sort	—	—	—

Table 2: Running time in microseconds.

The dashes above will be replaced with the actual running times from my program after execution.

5. Checking the Sorted Arrays

The program prints the first 10 elements of every sorted array for each array size. These values are used to check that every sorting algorithm produces the array in increasing order.

Algorithm	1,000	5,000	10,000
Bubble Sort	—	—	—
Insertion Sort	—	—	—
Selection Sort	—	—	—
Quick Sort	—	—	—
Merge Sort	—	—	—
Heap Sort	—	—	—

Table 3: First 10 elements after sorting.

The actual values will be taken from the execution output.

6. Time Complexity

Bubble Sort, Insertion Sort, and Selection Sort grow like $\Theta(n^2)$ in the worst case.

Merge Sort and Heap Sort grow like $\Theta(n \log n)$.

Quick Sort has an average running time of $\Theta(n \log n)$, but its worst case is $\Theta(n^2)$. Since the program uses the last element as the pivot, an already sorted or reversed array can produce the worst case for Quick Sort.

As the input size increases, the algorithms with quadratic growth should require much more work than the algorithms with $n \log n$ growth.

7. Comparison of Measurements with Theory

The timing results from the program will be compared with the expected growth rates.

For the smaller input size, the difference between the algorithms may not be very large because the arrays are still relatively small. For larger input sizes, the difference between quadratic and $n \log n$ growth should become more noticeable.

The actual measurements are affected by the computer, compiler, background processes, and other system activity. Therefore, the timing results are used together with the theoretical time complexities rather than as exact mathematical values.

8. Choice of Algorithm

For a large random array, an algorithm with $\Theta(n \log n)$ growth is more suitable than an algorithm with $\Theta(n^2)$ growth because the amount of work increases more slowly as the input size becomes large.

For an array that is already almost sorted, Insertion Sort can be useful because it only needs to move elements that are out of order. Bubble Sort with the early stopping condition can also take advantage of an already sorted or nearly sorted array.

The final choice also depends on the requirements of the application, such as memory usage and the behavior of the input data.

9. Execution Snapshot

The execution screenshot of the C++ program is included below.

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$ g++ -std=c++17 Comparing6algorithms.cpp -o Comparing6algorithms.exe
./Comparing6algorithms.exe
Comparing it now
Bubble Sort - A: 25 Sorted: 7 Reversed: 28
Insertion Sort - A: 20 Sorted: 7 Reversed: 28
Selection Sort - A: 28 Sorted: 28 Reversed: 28
Quick Sort - A: 14 Sorted: 28 Reversed: 28
Merge Sort - A: 17 Sorted: 12 Reversed: 12
Heap Sort - A: 26 Sorted: 27 Reversed: 24

```

Figure 12: Output 1

```

Array size: 1000
Bubble Sort: 1497 microseconds
First 10: 168 239 314 680 1098 1187 1323 1391 1498 1760
Insertion Sort: 516 microseconds
First 10: 168 239 314 680 1098 1187 1323 1391 1498 1760
Selection Sort: 1074 microseconds
First 10: 168 239 314 680 1098 1187 1323 1391 1498 1760
Quick Sort: 69 microseconds
First 10: 168 239 314 680 1098 1187 1323 1391 1498 1760
Merge Sort: 174 microseconds
First 10: 168 239 314 680 1098 1187 1323 1391 1498 1760
Heap Sort: 89 microseconds
First 10: 168 239 314 680 1098 1187 1323 1391 1498 1760

Array size: 5000
Bubble Sort: 37411 microseconds
First 10: 3 4 17 31 169 173 206 289 293 294
Insertion Sort: 12320 microseconds
First 10: 3 4 17 31 169 173 206 289 293 294
Selection Sort: 25871 microseconds
First 10: 3 4 17 31 169 173 206 289 293 294
Quick Sort: 515 microseconds
First 10: 3 4 17 31 169 173 206 289 293 294
Merge Sort: 919 microseconds
First 10: 3 4 17 31 169 173 206 289 293 294
Heap Sort: 544 microseconds
First 10: 3 4 17 31 169 173 206 289 293 294

```

Figure 13: Output 2

```
Array size: 10000
Bubble Sort: 181938 microseconds
First 10: 1 9 19 22 26 45 70 82 114 116
Insertion Sort: 51636 microseconds
First 10: 1 9 19 22 26 45 70 82 114 116
Selection Sort: 93273 microseconds
First 10: 1 9 19 22 26 45 70 82 114 116
Quick Sort: 940 microseconds
First 10: 1 9 19 22 26 45 70 82 114 116
Merge Sort: 1920 microseconds
First 10: 1 9 19 22 26 45 70 82 114 116
Heap Sort: 1182 microseconds
First 10: 1 9 19 22 26 45 70 82 114 116

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment#01_machado.cpp
$
```

Figure 14: Output 3